

RELAZIONE DEL PROGETTO SISTEMI OPERATIVI



Swaran Singh

7159864

Sommario

Sommario	2
1. INTRODUZIONE E SPECIFICHE DI PROGETTO	3
1.1 Obiettivi del sistema	3
1.2 Requisiti funzionali e divisione dei ruoli	3
2. ARCHITETTURA DEL SISTEMA	3
2.1 Modello Multiprocesso (Forking Server)	3
2.2 Ciclo di vita del processo figlio	4
2.3 Gestione dei processi zombie	4
3. PROTOCOLLO DI COMUNICAZIONE E LOGICA DI RETE	4
3.1 Struttura dei pacchetti (Header e Payload)	4
3.2 Gestione della lettura sul socket	5
3.3 Allineamento dei dati	5
4. PERSISTENZA DEI DATI	5
4.1 Organizzazione dei record su file binari (.dat)	5
4.2 Meccanismo di Locking tramite fcntl	5
4.3 Gestione delle richieste di prenotazione	6
5. MANUALE UTENTE E GUIDA ALLA COMPILAZIONE	6
5.1 Ambiente di sviluppo	6
5.2 Struttura del Makefile	6
5.3 Popolamento iniziale del database	7
5.4 Interazione del client tramite Opcode	7

1. INTRODUZIONE E SPECIFICHE DI PROGETTO

1.1 Obiettivi del sistema

La presente relazione descrive la progettazione e l'implementazione di un'infrastruttura client-server orientata alla gestione e alla consultazione di un Booking System distribuito. L'obiettivo primario dell'applicazione è consentire la prenotazione concorrente di risorse condivise (quali aule didattiche, postazioni di laboratorio o sale riunioni) garantendo al contempo due requisiti fondamentali dei sistemi informativi moderni: la consistenza atomica dei dati memorizzati e il rispetto della riservatezza delle informazioni personali degli utenti.

Il sistema sfrutta le API dei socket POSIX nel dominio Internet (AF_INET) con protocollo orientato alla connessione (TCP), garantendo un canale di trasporto affidabile e privo di perdite.

1.2 Requisiti funzionali e divisione dei ruoli

L'infrastruttura prevede una netta separazione dei privilegi operativi, implementata attraverso tre livelli di autorizzazione:

- **GUEST (Utente non autenticato):** Ha un accesso limitato alle sole funzioni di registrazione, autenticazione e visualizzazione dell'elenco pubblico delle aule disponibili.
- **USER (Utente standard autenticato):** Può inviare nuove richieste di prenotazione per una specifica risorsa, indicando data, ora di inizio e numero di slot temporali richiesti (dove ogni slot equivale a un'ora). È abilitato a consultare esclusivamente le proprie prenotazioni per ovvie ragioni di privacy.
- **SUPERUSER (Amministratore):** Detiene il controllo completo sul sistema. Può creare nuove aule, visualizzare l'elenco globale di tutte le prenotazioni memorizzate nel sistema, approvare o rifiutare le richieste pendenti e forzare manualmente lo stato di un record in caso di necessità amministrative.

2. ARCHITETTURA DEL SISTEMA

2.1 Modello Multiprocesso (Forking Server)

Per garantire la massima scalabilità e impedire che le richieste bloccanti di un singolo client possano penalizzare le prestazioni globali dell'infrastruttura, il Server adotta un modello architetturale multiprocesso.

Il processo principale (Padre), dopo aver eseguito le fasi di inizializzazione e binding, entra in un ciclo infinito all'interno del quale invoca la chiamata di sistema `accept()`. All'arrivo di una nuova connessione, il server esegue una copia di se stesso tramite `fork()`:

- Il processo Padre chiude immediatamente il descrittore del socket connesso restituito da `accept` e si rimette in ascolto per nuove richieste.
- Il processo Figlio eredita i descrittori del file del padre (chiude quelli non necessari) e assume la responsabilità esclusiva di servire il client associato.

2.2 Ciclo di vita del processo figlio

Per mantenere il codice pulito e modulare, il processo figlio esegue una mutazione della propria immagine di memoria tramite la chiamata di sistema `exec1()`. Prima di invocare la `exec`, il figlio esegue una duplicazione del descrittore del socket del client sul descrittore standard prefissato nella configurazione. Successivamente viene caricato l'eseguibile separato chiamato `./server_child`. Questo approccio permette di separare la logica di gestione della rete e dell'ascolto (svolta dal demone principale) dalla logica applicativa di parsing dei comandi del client, ottimizzando la manutenzione del codice sorgente.

2.3 Gestione dei processi zombie

La terminazione asincrona dei processi figli comporta la generazione del segnale `SIGCHLD` inviato al processo padre. Se il padre non raccoglie lo stato di terminazione dei figli tramite una chiamata `wait()` o `waitpid()`, questi ultimi rimangono nel sistema sotto forma di processi zombie. Per ovviare a questo problema è stato deciso di accettare e ignorare il `SIGCHLD`.

3. PROTOCOLLO DI COMUNICAZIONE E LOGICA DI RETE

3.1 Struttura dei pacchetti (Header e Payload)

La comunicazione tra client e server è regolata da un protocollo a livello applicazione rigido, strutturato in due parti distinte per evitare ambiguità nella lettura dei flussi TCP.

Ogni messaggio è composto da:

1. **Header (Dimensione fissa - 8 byte):** Contiene un intero (`OPCODE`) che identifica il codice dell'azione richiesta (es. `OPCODE_LOGIN`, `OPCODE_CREATE_BOOKING`) e un secondo intero (`payload_size`) che specifica la dimensione esatta del blocco di dati a seguire.
2. **Payload (Dimensione variabile):** Contiene la struttura dati necessaria per completare l'operazione (es. `struct Booking`, `struct LoginCredentials`).

3.2 Gestione della lettura sul socket

I canali di rete TCP possono trasmettere i pacchetti in modo frammentato. Di conseguenza, una singola chiamata a `read()` potrebbe restituire un numero di byte inferiore rispetto a quelli attesi. Per ovviare a questo problema è stata implementata la funzione `read_exact()`, che cicla fino al completamento della dimensione richiesta.

3.3 Allineamento dei dati

Le architetture hardware dei computer possono adottare convenzioni differenti per la memorizzazione dei byte in memoria (Big-Endian vs Little-Endian). Per consentire l'operabilità del Booking System in entrambi gli ambienti, tutte le informazioni numeriche composte da più byte trasmesse sulla rete vengono convertite nel formato standard di rete (Network Byte Order, Big-Endian) prima dell'invio, e riconvertite nel formato nativo dell'host (Host Byte Order) alla ricezione.

4. PERSISTENZA DEI DATI

4.1 Organizzazione dei record su file binari (.dat)

La persistenza dello stato del sistema è affidata a tre file binari: `users.dat`, `rooms.dat` e `bookings.dat`. L'uso di strutture a dimensione fissa (es. `struct Booking`) permette di calcolare lo scostamento esatto di un record all'interno del file ed effettuare aggiornamenti in posizioni specifiche tramite la funzione `fseek()`.

4.2 Meccanismo di Locking tramite `fcntl`

Poiché il server opera tramite processi figli indipendenti che accedono concorrentialmente agli stessi file di database, è indispensabile regolare l'accesso

per evitare corruzioni o sovrascritture dei dati. Si ricorre alla chiamata di sistema `fcntl()`, che implementa i lock a livello di file system. Vengono definite due tipologie di lock:

- **Lock in Lettura (Condiviso):** Permette a più processi figli di leggere contemporaneamente lo stato dei file, bloccando i tentativi di scrittura.
- **Lock in Scrittura (Esclusivo):** Garantisce l'accesso esclusivo a un singolo processo figlio per le fasi di inserimento o modifica, sospendendo qualsiasi altra operazione di lettura o scrittura.

4.3 Gestione delle richieste di prenotazione

Il sistema adotta un approccio basato sull'approvazione differita da parte di un amministratore. È consentito a più utenti di inoltrare richieste di prenotazione (in stato PENDING) per la medesima risorsa anche in slot temporali sovrapposti. La risoluzione delle collisioni è delegata al server: quando l'amministratore approva una specifica richiesta, la logica applicativa individua automaticamente e rifiuta tutte le altre prenotazioni pendenti che presentano una sovrapposizione temporale per quella stessa risorsa.

5. MANUALE UTENTE E GUIDA ALLA COMPILAZIONE

5.1 Ambiente di sviluppo

L'intero progetto è stato sviluppato in un unico ambiente di programmazione utilizzando

- Windows 11
- WSL 2.6.3 con Ubuntu 24
- gcc 13.3

5.2 Struttura del Makefile

La compilazione del progetto è interamente automatizzata tramite un file makefile compatibile con l'utility make. Il file definisce le dipendenze incrociate tra i moduli e compila i tre binari principali (`server`, `client`, `server_child`) e l'eseguibile di utility `populate`.

I comandi principali sono:

- `make`: Esegue la compilazione progressiva di tutti i componenti del sistema.
- `make clean`: Rimuove i file oggetto e i file binari generati precedentemente.
- `make populate`: Esegue la compilazione per popolare il database.
- `make clean-data`: Elimina i database binari (`.dat`) presenti nella cartella di lavoro per resettare lo stato del sistema.

5.3 Popolamento iniziale del database

Per testare il sistema in condizioni realistiche è stato implementato il programma `populate.c`. Una volta eseguito tramite il comando `./populate`, il programma provvede a generare dummy data (stanze, utenti, e prenotazioni).

5.4 Interazione del client tramite Opcode

L'interfaccia utente del Client è basata su un terminale interattivo testuale molto intuitiva. All'avvio, il programma legge i parametri dal file di configurazione `.client_settings` ed effettua la chiamata `connect()`.

L'utente interagisce inserendo direttamente i codici numerici dei comandi:

Il client convalida l'input localmente verificando che l'opcode inserito sia compatibile con il livello di autorizzazione corrente dell'utente prima di inoltrare la richiesta sulla rete.